
Source Auditing Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *October 17th, 2019*

Prerequisites and Course Map

1. Assumed knowledge (not taught here; get it elsewhere first):

- (a) C fluency: pointers, integer promotion rules, struct layout and padding, common initial sequence rules, varargs. You must be able to write the code you audit.
- (b) One assembly language at reading level and its calling conventions (the binary RE module covers the mapping, but you need one to start).
- (c) Syscall literacy: what `open/read/mmap/fork/exec` actually do, what a descriptor is, enough signal semantics to recognize what a handler may not do.
- (d) Build systems: static vs. dynamic linking, configuration switches (`-enable/-without`) and how they map to compiled-in code (the Attack Surface section leans hard on this).
- (e) Python for tooling and one scripting interface to a debugger.

2. Reading order (each module assumes its predecessors):

- (a) **Source auditing:** the bug patterns, the mitigation map, and the triage pipeline are referenced by every later module. Do its lab (AppWeb, heap.c) before anything else.
- (b) **Fuzzing:** consumes the bug classes and the mitigation map; produces crash sets which its triage section converts into reports.
- (c) **Binary RE:** the no-source arm of static analysis; its examples reuse the same CVEs cited in source auditing.
- (d) **Debugging:** the dynamic-analysis craft the other modules keep alluding to; teach it after source auditing so the bug-pattern shapes are known, before exploitation.
- (e) **From primitive to exploit:** assembles the mitigation map's bypass column into techniques (code reuse, infoleak, allocator grooming).
- (f) **Linux kernel auditing:** the capstone — all previous skills at kernel privilege, plus concurrency and elevation specifics.
- (g) **Linux and Android security models:** the enforcement layer every primitive above is priced against (credentials, capabilities, LSM/MAC, namespaces, seccomp; the Android uid sandbox, permission model, SELinux domains, Binder, verified boot); its confused-deputy and check-vs-use checklists are the authorization-side counterpart of this module's bug patterns.

3. Each module pairs one lecture with one lab; the lab deliverable always ends in a written report in exactly the **From Bug to Report** format, because that report is the profession's unit of work.

Introduction

1. A primary skill of source code auditing is identifying which code may be vulnerable and is reachable from an exposed attack surface. Experienced code auditors are continually asking themselves the following questions:
 - (a) "Where should the limited time auditing be spent to maximize the chances of bug discovery?"
 - (b) "What pieces of code should be checked thoroughly and which can be ignored?"
 - (c) "Upon finding a bug, should more time be spent exploring to find more bugs, or should the focus turn to exploitation of the current set of bugs? (Exploration versus Exploitation Tradeoff).
2. Large code bases make these questions difficult to answer. However, an experienced auditor will perform a thorough attack surface analysis and follow the logical flow of attacker data understanding and verifying each use of the data. General guidance to follow when starting any security audit is:
 - (a) Learn to read and write target language (C, C++, PHP, Java, etc).
 - (b) Understand the architecture in which your target will be executing (ARM, ARM64, x86, x86-64, risc-v, etc)
 - (c) Know common bug patterns in the target language.
 - (d) Research existing CVEs and public research about the target.
 - (e) Find code which is in dis-use or rarely executed.
 - (f) Code is built in layers or modules. Scope the problem by looking at layers or modules individually and studying the API's.
 - (g) Find the ages of the code layers, bugs tend to be in the most recent or very old code.
 - (h) Find large back-end components of the software such as memory allocators, parsing functions/libraries, JIT engines, garbage collectors, etc.
 - (i) Understand the purpose of the code, what the author was trying to do and how the implementation falls short.
 - (j) Find the most complicated pieces of code with high cyclomatic complexity. Data structures which are used frequently or are polymorphically used (cast/unions) are important to understand.
 - (k) Get a feel for the code style and find places the style is violated. This can tell you if multiple authors are writing and merging code and opens up chances for mis-communications.
 - (l) Need to know how high level code is translated into assembly looking for when the intended logical flow does not match the implementation.

- (m) Track down documentation
 - i. RFC's
 - ii. User manuals
 - iii. Client software
 - iv. Source leaks (even out of date code is valuable)
 - v. Bug trackers
 - vi. Code repositories/logs
 - vii. Stalk/blame bad developers
 - viii. Read all comments in source
 - ix. Find prior CVEs in public databases (exploitdb, cvedetails, etc)

Comparison of VR Techniques

1. Searching for vulnerabilities can be done using static or dynamic analysis. Static analysis includes source code auditing and binary reverse engineering. Dynamic analysis includes fuzz testing and debugging among other more advanced techniques.
2. Dynamic analysis
 - (a) Very few false positives (flagging bugs that don't exist)
 - (b) False negative rate (missed bugs) can be reduced by adding code coverage, feedback fuzzing or memory sanitizers
 - (c) Scaling can be done by adding more processing time
 - (d) Progress can be made using blackbox methods
 - (e) Able to find non-obvious bugs which utilize code from disparate parts of the source code base. This can make root causing hard.
 - (f) Special instrumentation is needed to find bugs that do not result in a crash.
3. Static analysis:
 - (a) Many more false positives (flagging bugs that don't exist) especially with automated tools
 - (b) False negative (missed bugs) rate is a function of the auditors understanding of the attack surface and code base
 - (c) Requires considerable computational/human resources to scale
 - (d) Progress requires considerable internal target knowledge
 - (e) Bugs found here typically include code which is localized to one area of the target.
 - (f) Source auditing can find non-memory corruption bugs (non-crashing) which are missed by fuzzing techniques.

Attack Surface Analysis

1. Attack surface analysis is the process of identifying the points of entry for any untrusted data. Knowing where incoming data is processed will dictate how an attack scenario proceeds. Attack surface analysis is valuable to source auditing to answer, "Where do I start auditing?", "What code is accessible and what code can be ignored?", "Has this code been audited or fuzzed before?", and "How many vulnerabilities have been found in this component before? Are there any left?".
2. Program input vectors are:
 - (a) network sockets (BSD-style sockets)
 - (b) inter-process communication (UNIX domain sockets, binder, signals, shared memory)
 - (c) memory mapped pages from the operating system (think dirty-cow)
 - (d) files on the filesystem
 - (e) environment variables
 - (f) software or firmware update channels
3. An important thing to keep in mind when auditing is what options were used at compile time. This will help you ignore code not compiled into the target.
4. Network servers or operating systems are highly configurable and some code may be present in the source base, but not compiled into the running target binary
5. Software such as webserver are highly extensible meaning other software modules may or may not be present. These modules are sometimes more vulnerable because they are written by third parties.
6. Some software has features available in debug builds which are not present in the release and the opposite is true as well. For example, assert statements in Javascript engines are compiled out in release builds.

Bug Patterns Through Small Examples

1. Naive approach to source auditing is to find all the memcpy's, strcpy's or other patterns and then trace the parameters back to their origin. This can be effective for smaller targets, but is not sufficient and untenable for larger targets.
2. It is better to start from the input vector and read through the code which handles the input data. Trace the data and examine the data validation code looking for unsanitized input by ensuring all data is constrained properly.

The Primitive Vocabulary

1. Before walking the examples, fix the vocabulary. Every memory-corruption bug is described by the primitive it grants an attacker, never by the bug's name:
 - (a) **Arbitrary read/write**: the attacker names the address. The strongest primitive; either one alone is usually game over.

- (b) **Relative read/write:** the attacker controls an offset added to a base they know (a heap chunk, a stack slot, a mapping). Usually paired with a leak that converts it into an absolute one.
 - (c) **Controlled-length write:** the attacker controls how many bytes are written but not the content or the target.
 - (d) **Controlled-content write:** the attacker controls what is written but not where or how much.
 - (e) **Lifetime control:** the attacker controls when an object is freed or reused (the UAF and double-free family).
 - (f) **Information disclosure:** the attacker reads memory they should not (the leak primitive every ASLR bypass needs).
2. The audit method for every pattern below is the same three questions:
- (a) Where do the length, size, index, and pointer arguments come from, and are they validated against the destination's true size?
 - (b) What does the code assume about the value after the check that an attacker can break (signedness, truncation, aliasing, lifetime)?
 - (c) What primitive does the resulting corruption actually grant (the vocabulary above)? Only the third answer separates a crash from a finding.
3. The examples below are deliberately small. The skill being taught is recognizing the shape in a large code base, not solving the toy.

Stack Overflow

1. The classic stack overflow is simple but illustrates the fundamental reason overflows are exploitable. Stacks store in-line metadata which is critical to the control flow of a program (i.e. function return addresses and stack frame pointers).
2. Stacks have been exhaustively studied and modern stack overflows are hard to exploit due to FORTIFY_SOURCE protections inserted by the compiler. Stack cookies, stack variable re-ordering, safe string handling functions (think strncpy, snprintf, etc) and moving libraries to non-ASCII address ranges are all mitigations to prevent exploitation.
3. Exploitation can still occur if interesting local variables exist on the stack and are referenced before the stack cookie is checked.
4. Noticing statically sized stack buffers is useful in finding such bugs. In addition, mis-use of the stack allocation routine alloca() can lead to a stack overflow.
5. A stack overflow is different from stack exhaustion. Stack exhaustion occurs when the stack runs out of space due to large allocations or deep function recursions.
6. One interesting strategy that continues to be exploitable is if you can write bytes to the stack at predictable offsets. If so, then overwriting pointers on the stack is useful to shift stack frame pointers or pointers to objects on the stack up or down into perhaps attacker controlled areas.

7. The following example of a stack corruption bug is based on CVE-2018-7445. Note that the `alloca()` (in Linux) will allocate a buffer on the stack by subtracting the size passed in from the stack pointer. This bug pattern is not possible in Windows because the `alloca` implementation will allocate a single page at a time and dereference each page incrementally.

Listing 1: Stack Corruption

```
1 void read_contents(int fd, unsigned int content_length_value)
2 {
3     unsigned char *stack_buffer;
4
5     // consider what happens when content_length_value = -1
6     stack_buffer = alloca(content_length_value);
7     read(fd, stack_buffer, content_length_value);
8 }
```

Heap Overflow

1. Exploiting a heap overflow requires knowing the implementation details. The implementation details will dictate the exploitation approach and what exploitation mitigations are present.
2. All heap implementations follow the POSIX API or `malloc()`, `calloc()`, `realloc()`, `free()`, etc. There are many heap allocators:
 - (a) `dlmalloc` (general purpose)
 - (b) `ptmalloc2` (glibc)
 - (c) `jemalloc` (FreeBSD, Firefox)
 - (d) `tcmalloc` (Google)
 - (e) `libumem` (Solaris)
3. Ideal heap vulnerabilities consist of several qualities of which a subset may be present.
 - (a) Control of the heap allocation size
 - (b) Control of the data overflowing the buffer
 - (c) Control of the length overflowed out-of-bounds
 - (d) Control of the life-time of the allocation (able to invoke when allocation and free occur)
 - (e) Ability to allocate and groom the heap with interesting objects containing meta-data (i.e. function pointers)
 - (f) Ability to invoke the use of useful meta-data that is overflowed.
4. The following example illustrates a function with an obvious heap overflow in the `CREATE` command. This simple example illustrates control of allocation size, length of overflow, life-time. This bug would require another heap primitive to overflow into containing metadata. In addition, the overflowed data is ASCII limiting null bytes.

Listing 2: Heap Corruption

```
1 #define MAX_STRS 0x100
2 #define STRLEN_MAX 0x100
3 enum cmd { CREATE = 0x10, DELETE, FIND };
4 char *g_strs[MAX_STRS];
5 int n_strs= 0;
6
7 char *find(char *str)
8 {
9     for(int i = 0; i < n_strs; i++) {
10         if (!strcmp(str, g_strs[i])) return g_strs[i];
11     }
12     return NULL;
13 }
14
15 // called repeatedly with user controlled command and c-string
16 char* str_store(enum cmd c, char *str)
17 {
18     char *ret = NULL;
19     switch (c) {
20         case CREATE:
21             if (n_strs < MAX_STRS) {
22                 g_strs[n_strs] = malloc(STRLEN_MAX);
23                 strcpy(g_strs[n_strs], str);
24                 ret = g_strs[n_strs++];
25             }
26             break;
27         case DELETE:
28             free(find(str));
29             break;
30         case FIND:
31             ret = find(str);
32             break;
33     }
34
35     return ret;
36 }
```

Format String Bugs

1. Format strings are not found often, because they are easy for compilers to programmatically check for. The compiler can easily check if the format string is not a constant string and can also check if the number of parameters in the format string matches the number and type of arguments. This prevents the class of bug entirely.
2. Also, FORTIFY_SOURCE features of gcc make these bugs hard to exploit. However, there are automated libraries for exploiting these bugs now.
3. If the attacker has control of the format string passed to printf, fprintf, vsprintf,

sprintf, etc, then he can read memory OOB, dereference memory OOB, and with %n feature can write attacker controlled data OOB.

4. The %n is widely disabled these days.

Listing 3: Format Strings

```
1 // based on CVE-2012-0809
2 void format_string(char *user_controlled)
3 {
4     int foo = 1;
5
6     // very bad, what would "%s", "%n" or "%x" do?
7     printf(user_controlled);
8
9     // what does the "%s" do?
10    printf("User_string_is_%s");
11
12    // what does the second "%x" print?
13    printf("User_variable_is_var=%x%x", foo);
14 }
```

Out-of-bounds (OOB) Read/Write

1. These can be caused by unchecked offsets, and can be relative or absolute.
2. A relative read/write is when the offset is added to a pointer dynamically allocated.
3. If the offsets are signed, then you can read/write relative before the buffer. If not, you can only write after. This will affect your memory grooming technique used.
4. An absolute read/write is when the attacker controls the dereferenced pointer being written to (i.e. can write to any address specified directly by the attacker).

Listing 4: Out-of-bound Read

```
1 #define MAX_SIZE 0x1000
2 char *g_buf;
3 void oob(uint8_t *buffer, uint32_t len, int32_t seek)
4 {
5     if (len > MAX_SIZE) return;
6
7     if (!g_buf) {
8         g_buf = malloc(len);
9         if (!g_buf) return;
10    }
11
12    // what happens if user controls seek?
13    memcpy(g_buf, buffer + seek, len - seek);
14 }
```

Arbitrary free

1. If the attacker can control the pointer value passed to free.
2. This can often times be turned into a use-after-free (UAF) by free'ing memory still in use.

Listing 5: Arbitrary Free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name) {
8     // set list name, what does this overwrite?
9     Obj *o = malloc(sizeof(Obj));
10    o->next = NULL;
11    strcpy(o->buf, name);
12
13    Obj *c = &list_head;
14    while(c->next)
15        c = c->next;
16    c->next = o;
17 }
18
19 void free_list() {
20     Obj *c = list_head.next;
21     do {
22         Obj *t = c;
23         c = c->next; // advance pointer
24         free(t); // free current
25     } while(c);
26 }
```

Double Free

1. A double free will add the same pointer to the heap allocator's free list twice.
2. Then two new allocations will be served the same free memory block which could result in a type confusion.
3. Most libc implementations will catch this because it's a simple check.

Listing 6: Double Free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name)
8 {
9     // setup object
10    Obj *o = calloc(1, sizeof(Obj));
11    strncpy(o->buf, name, 0xff);
12    o->next = NULL;
13
14    // add to list
15    Obj *c = &list_head;
16    while(c->next)
17        c = c->next;
18    c->next = o;
19 }
20
21 void remove_object(char *name)
22 {
23     Obj *c = list_head.next;
24     while(strncmp(c->buf, name, 0xff)) {
25         c = c->next;
26     }
27
28     // what does this case forget to do?
29     if (c) {
30         free(c);
31     }
32 }
33
34 void free_list()
35 {
36     Obj *c = list_head.next;
37     do {
38         Obj *t = c;
39         c = c->next; // advance pointer
40         free(t); // free current
41     } while(c);
42 }
43
44 int main()
45 {
46     // what is the state of memory after these calls?
47     add_object("foo");
48     remove_object("foo");
49
50     // what happens when we free the list?
51     free_list();
52 }
```

Use-after-free (UAF)

1. A use-after-free (UAF) is problematic because an attacker may be able to replace the memory.
2. Critical aspects of exploitability of these bugs are the ability to spray or groom the heap to replace the memory with attacker controlled data. Then having that attacker controlled way used in an interesting way.

Listing 7: Use-after-free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name) {
8     Obj *o = calloc(1, sizeof(Obj));
9     strncpy(o->buf, name, 0xff);
10    o->next = NULL;
11
12    Obj *c = &list_head;
13    while(c->next) c = c->next;
14    c->next = o;
15 }
16
17 void remove_object(char *name) {
18     Obj *c = list_head.next;
19     while(strncmp(c->buf, name, 0xff)) c = c->next;
20
21     // what does this case forget to do?
22     if (c) free(c);
23 }
24
25 void print_list() {
26     Obj *c = &list_head;
27     while(c->next) {
28         printf("%s\n", c->buf);
29         c = c->next;
30     }
31 }
32
33 int main() {
34     // what is the state of memory after these calls?
35     add_object("foo");
36     add_object("bar");
37     remove_object("foo");
38
39     // what happens when we do this?
40     print_list();
41 }
```

Integer Overflow Example

1. The following code snippet is inspired by CVE-2015-3864 which is the Android StageFright bug in the MP4 parser.
2. The inputs to the function below are attacker controlled. Therefore, the addition and multiplication can overflow the size of a 32 bit unsigned integer. This results in the allocation of fewer bytes than necessary.
3. The memcpy will then copy data into the heap. Notice that the in_buf_size parameter is large to hit the overflow meaning the memcpy will copy large amounts of data and most likely segfault the process when attempting to write to unallocated/unmapped pages.
4. Many times one bug type can be transformed into another bug type. Here an integer overflow enabled a heap overflow.

Listing 8: Integer Overflow Code

```
1 void copy_buffer(unsigned char *in_buf, unsigned len)
2 {
3     unsigned char *buffer;
4     unsigned int buf_size;
5
6     buf_size = len + 16; // integer overflow
7     buffer = (unsigned char*)malloc(buf_size *
8                                     sizeof(unsigned char));
9     if (buffer != NULL) {
10         memcpy(buffer, in_buf, len);
11     }
12 }
```

Signedness Issues

1. Signedness issues can occur when an attacker can influence an integer type to be negative.
2. It is important that the developers are aware of integer casting rules to prevent these issues.

Listing 9: Signedness Issues

```
1 void sample(unsigned char *in_buf, int len)
2 {
3     if (len < 0) return;
4
5     // what happens if len == 0?
6     for(int i = len-1; i != 0; --i) {
7         in_buf[i] = to_upper(in_buf[i]);
8     }
9 }
```

Type Confusions

1. Type confusions are more often found in languages where polymorphism is built in such as C++.
2. However, type confusion can occur where type casting occurs. Look out for explicit cast operators in C and reinterpret_type() calls in C++. These circumvent type safety semantic checking in the compilers.

Listing 10: Type Confusion

```
1 #define NAME_TYPE 1
2 #define ID_TYPE 2
3
4 struct MessageBuffer
5 {
6     int msgType;
7     union {
8         char *name;
9         int nameID;
10    };
11 };
12
13 int main (int argc, char **argv) {
14     struct MessageBuffer buf;
15     char *defaultMessage = "Hello World";
16
17     buf.msgType = NAME_TYPE;
18     buf.name = defaultMessage;
19
20     if (argc > 1) {
21         buf.msgType = atoi(argv[1]);
22         buf.name = argv[2];
23     }
24
25     // what happens when the wrong type is given?
26     if (buf.msgType == NAME_TYPE) {
27         printf("Message: %s\n", buf.name);
28     }
29     else {
30         printf("Message: Use ID %d\n", buf.nameID);
31     }
32
33     return 0;
34 }
```

Information Leaks/Disclosure

1. The most infamous information disclosure is HeartBleed (CVE-2014-0160) in OpenSSL which would allow a remote attacker to dump heap data from a server process.
2. It was called HeartBleed because the heartbeat (hello I'm alive) packet in the TLS was poorly implemented in the library.

3. This is an example of when a bug isn't found because the attack surface goes unnoticed. Typically new attack surfaces (no matter how simple) can harbor bugs.

Listing 11: Info Leak

```
1 // based on CVE-2014-0160, the attacker controls payload
2 // so it can be increased beyond the size of pl and returned
3 // to the attacker over the socket.
4 typedef struct payload {
5     char pl[0x10];
6 };
7 struct payload g_pl;
8 void send_reply(int fd, int payload)
9 {
10     char *bp = malloc(payload);
11     memcpy(bp, g_pl.pl, payload);
12     write(fd, bp, payload);
13 }
```

Command Injection

1. Command injection comes from using unsanitized input. When calling out to system via system(), popen(), execve(), etc, any shell escape characters need to be filtered out.

Listing 12: Command Injection

```
1 void unlink_file(char *fname)
2 {
3     char buf[1024];
4     struct stat statbuf;
5
6     // get file stats
7     if (!stat(fname, &statbuf)) return;
8
9     // check if regular file
10    // BUG: == binds tighter than \& - what does this actually test?
11    if (statbuf.st_mode & S_IFMT == S_IFREG) {
12        // what happens if fname has shell escape characters is in it?
13        snprintf(buf, 1024, "/bin/rm_%s", fname);
14
15        FILE *fp = popen(buf, "r");
16        if (fp == NULL) return;
17        pclose(fp);
18    }
19 }
```

Race Condition

1. Race conditions, also known as TOCTOU (Time-Of-Check to Time-Of-Use), are bugs in concurrent programs (multi-threading or multi-process) where shared state is not properly synchronized with the use of locking mechanisms.

2. These are hard to check for using static analysis techniques such as source code auditing.
3. When you see locks being used or state being shared, ask "What memory and variables are shared?" and "What synchronization mechanisms are used to serialize access (spin locks, event locks, reader/writer locks, semaphores)?"
4. Some race conditions are very hard to hit. Also look for global locks which will be in higher function calls. These are sometimes used as band-aids for the developers when run time is not an issue. Past Linux versions and Python2 used global locks at runtime.
5. Dirty Cow (CVE-2016-5195) is a very infamous race condition but too elaborate to go into here.

Listing 13: Race Condition

```
1 // what happens if we run this and have another process
2 // switch out the file after the check?
3 void racer(unsigned char *fname)
4 {
5     if (access(fname, R_OK | W_OK) != 0) {
6         perror("file_permission_denied");
7         return;
8     }
9
10    // read and write file
11 }
```

Logic Error

1. Logic error due to a bad code merge or copy and paste error.
2. Permits MitM attacker to capture and modify data in sessions protected by SSL/TLS by short circuiting the SHA1 validation.

Listing 14: Logic Error

```
1 void SSLVerifySignedServerKeyExchange()
2 {
3     // <snip'ed code>
4     ashOut.data = hashes + SSL_MD5_DIGEST_LEN;
5     hashOut.length = SSL_SHA1_DIGEST_LEN;
6     if ((err = SSLFreeBuffer(&hashCtx)) != 0)
7         goto fail;
8     if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
9         goto fail;
10    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
11        goto fail;
12    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
13        goto fail;
14    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
15        goto fail;
16    goto fail; // what does this line do?
17    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
18        goto fail;
19
20    err = sslRawVerify(...);
21    // <snip'ed code>
22 }
```

REF Counting

1. Reference counting can be viewed as a memory allocator with amortized work. All references to a single object use the same underlying structure/object memory. Allocation happens during initialization (RAII) and free'ing happens when reference count goes to zero.
 - (a) A put operation is decrementing (free'ing) a ref count. When refcount goes to zero, then memory is free'd.
 - (b) A get operation is incrementing (allocating) a ref count.
2. Mis-counting a refcount can lead to UAF.
 - (a) If an attacker can invoke an extra put, then a UAF can be directly achieved.
 - (b) If an attacker can invoke enough extra gets to integer overflow the refcount (wrapping it around to zero), the object can be prematurely free'd and a UAF achieved.

Listing 15: Incorrect Reference Counting

```
1 struct ref {
2     void (*free)(const struct ref *);
3     int count;
4 };
5
6 void ref_inc(const struct ref *ref)
7 {
8     ((struct ref *)ref)->count++;
9 }
10
11 void ref_dec(const struct ref *ref)
12 {
13     if (--((struct ref *)ref)->count == 0)
14         ref->free(ref);
15 }
16
17 // embed reference count in a struct/class
18 typedef struct Obj{
19     int data;
20     struct ref refcount;
21 } Obj;
22
23 // struct/class initialization
24 Obj *alloc_obj(int data) {
25     // allocate memory
26     Obj *o = calloc(1, sizeof(Obj));
27     o->refcount.free = free;
28
29     // init ref count
30     ref_inc(&o->refcount);
31
32     return o;
33 }
34
35 int main(int argc, char **argv)
36 {
37     if (argc != 2) return -1;
38     long num_to_alloc = strtol(argv[1], NULL, 10);
39
40     // what happens when num_to_alloc = 0xFFFFFFFF?
41     Obj *o = alloc_obj(42);
42     for(int i = 0; i < num_to_alloc; i++) {
43         ref_inc(&o->refcount);
44     }
45
46     // de-allocations
47     for(int i = 0; i < num_to_alloc; i++) {
48         ref_dec(&o->refcount);
49     }
50
51
52     return 0;
53 }
```

Uninitialized Memory Bugs

1. Use of uninitialized memory can be problematic if the attacker can replace the memory on the stack or heap with their data via a groom or spray.

Listing 16: Memory Used Uninitialized

```
1 #define ARRAY_SIZE 0x100
2 #define STR_SIZE 0x10
3 char **g_array;
4 void alloc_array(void) {
5     if (!g_array) {
6         g_array = malloc(ARRAY_SIZE * sizeof(char*));
7         if (!g_array) return;
8     }
9
10    for(int i = 0; i < ARRAY_SIZE; i++) {
11        g_array[i] = calloc(STR_SIZE, sizeof(char*));
12    }
13 }
14
15 void use_array() {
16     if (!g_array) return;
17     for(int i = 0; i < ARRAY_SIZE; i++) {
18         printf("%s\n", g_array[i]);
19     }
20 }
21
22 void free_array() {
23     if (!g_array) return;
24     for(int i = 0; i < ARRAY_SIZE; i++) {
25         free(g_array[i]);
26     }
27     g_array[0] = NULL; // is this sufficient?
28 }
29
30 int main() {
31     alloc_array();
32     free_array();
33     use_array();
34 }
```

Integer Promotion Bugs

1. Another good exercise which will help you improve your source auditing code is knowing about integer promotion rules. I recommend using the quiz at => pleasestopnamingvulnerabilities.com/integers.html.
2. As a rule of thumb, the compiler emit assembly code so that the integers are promoted to the widest type and then compared.
3. Integer ranges:

- (a) signed char: -128 to 127
- (b) unsigned char: 0 to 255
- (c) signed short: -32768 to 32767
- (d) unsigned short: 0 to 65535
- (e) signed int: -2147483648 to 2147483647

Listing 17: Integer Promotion

```
1 // Why does this give a strange, large integer number and not 255?
2 unsigned char x = 0;
3 unsigned char y = 1;
4 printf("%u\n", x - y);
5
6 // Why does this give "-1 is larger than 0"?
7 unsigned int a = 1;
8 signed int b = -2;
9 if(a + b > 0)
10     puts("-1 is larger than 0");
11
12 // Why does changing the type in the above example to short fix the problem?
13 unsigned short a = 1;
14 signed short b = -2;
15 if(a + b > 0)
16     puts("-1 is larger than 0"); // will not print
```

Truncation Bugs

1. Truncation is a special case of integer issues where a value silently passes through narrower types on its way between API layers. The value never goes negative and never overflows in the source sense — it is simply no longer the value the author thought it was.
2. The dangerous pattern is a size populated at a wide API boundary (a 64-bit `size_t`, a `u64` from a header field) that is narrowed when passed to internal helpers taking `int`, `u16`, or `u32`. Checks applied at the boundary no longer constrain the value after truncation: validating `claimed_len <= 0x10000` against a `u64` does nothing for a `u16` that can never hold `0x10000`.
3. Audit trick: find every narrowing cast/conversion between an attacker-controlled length and a `memcpy/read/recv` size, and re-evaluate each check against the narrowed value, not the original.

Listing 18: Truncation

```
1  /* verified: compiles cleanly at -O0; stated mismatch intended */
2  void download_chunk(int fd, uint64_t claimed_len)
3  {
4      unsigned char *out;
5      int len_i;
6      uint16_t len_16;
7
8      if (claimed_len > 0x10000) {
9          printf("too big\n");
10         return;
11     }
12     len_i = (int)claimed_len;
13     len_16 = (uint16_t)len_i; /* 0x10000 truncates to 0 */
14     out = malloc(len_16 ? len_16 : 1);
15     /* read length is up to 0x10000 into a buffer of len_16 bytes */
16     read(fd, out, len_i);
17     free(out);
18 }
```

Deserialization / Parser-Trust Bugs

1. Any format that carries both a type tag and a length (TLV structures, serialization frameworks, config blobs, kernel netlink messages) invites the same core bug family: the consumer of the parsed structure trusts properties that were never validated together.
2. The two classic dissociations: (a) the length validated was the container length, while the length used to copy is the element-claimed one; (b) the type tag was validated for membership in an enum, but the consumer dispatches on a tag that implies a data layout nobody validated (type confusion at the parser boundary).
3. Stagefright-class parsers, browser media stacks, and protocol dissectors are full of this class. Auditing method: for each (tag, length, data) triple, ask independently — who validated the tag? who validated the length? who validated that the layout implied by the tag matches the length?

Listing 19: Deserialization Type Confusion

```

1  /* verified: compiles cleanly at -O0; dissociations intended */
2  struct msg {
3      uint8_t type;
4      uint16_t len;
5      char data[];
6  };
7
8  /* wire format: [1-byte type][2-byte length][data] */
9  struct msg *parse_msg(const unsigned char *wire, size_t wire_len)
10 {
11     struct msg *m;
12     uint16_t len;
13
14     if (wire_len < 3) return NULL;
15     len = (wire[1] << 8) | wire[2];
16
17     /* which is checked: the on-wire length or the claimed length? */
18     if (len > wire_len - 3) return NULL;
19
20     m = malloc(sizeof(*m) + len);
21     if (!m) return NULL;
22     m->type = wire[0];
23     m->len = len;
24     memcpy(m->data, wire + 3, len);
25     return m;
26 }
27
28 /* consumer assumes 'type' determines the actual data layout */
29 void handle_msg(struct msg *m)
30 {
31     if (m->type == 1) {
32         /* data is a NUL-terminated string... is it? */
33         printf("name: %s\n", m->data);
34     } else if (m->type == 2) {
35         /* data is an array of exactly m->len 4-byte records... is it? */
36         uint32_t *recs = (uint32_t *)m->data;
37         for (int i = 0; i < m->len; i++)
38             printf("%u", recs[i]);
39         printf("\n");
40     }
41 }

```

Path and FD-Lifetime Races

1. Beyond the classic `access()` TOCTOU shown earlier, two related families: (a) path races where a `lstat/open` pair is separated by a rename or symlink swap in a writable directory; (b) fd-lifetime bugs where a descriptor validated once is later reused for a different file.
2. The audit question: between the moment a path/fd/user-controlled name is

checked, and the moment it is used, can some other thread or process substitute the object? Attack surface: any code running at elevated privilege writing to paths under user influence (home dirs, /tmp, spool, upload directories).

3. Kernel-adjacent variant: data structures holding a borrowed reference (struct file *) that another thread can close mid-operation.

Listing 20: Path Race

```
1  /* verified: compiles cleanly at -O0; pattern intended */
2  void append_log(const char *user_dir, const char *content)
3  {
4      char path[1024];
5      int fd;
6      struct stat st;
7
8      /* victim: privileged log write inside a user-influenced
9       * directory. lstat passes (regular file, not a symlink) ... */
10     snprintf(path, sizeof(path), "%s/latest.log", user_dir);
11     if (lstat(path, &st) != 0 || !S_ISREG(st.st_mode)) return;
12
13     /* ... but by the time open() runs, the user has replaced
14      * latest.log with a symlink. What closes this race? */
15     fd = open(path, O_WRONLY | O_APPEND);
16     if (fd >= 0) {
17         write(fd, content, strlen(content));
18         close(fd);
19     }
20 }
```

Signal-Safety and Error-Path Concurrency

1. Signal handlers execute as the interrupted thread: any libc function they call competes for locks the interrupted code may already hold. printf, malloc, and free are not async-signal-safe; the safe surface is write() and atomics/volatile only.
2. The related bug family: error paths and signal paths that free or mutate shared state concurrently with the main flow, causing double-frees or torn allocation state. In audits: for every early return, longjmp, or signal disposition, ask what cleanup runs and whether it is serialized with the main path.
3. This is the bridge between the Race Condition section and real-world allocator corruption: allocator state is a shared resource, and anything that touches it from an interrupt context is an immediate red flag.

Listing 21: Async-Signal-Unsafe Handler

```
1  /* verified: compiles cleanly at -O0; pattern intended */
2  static char *g_buf;    /* freed and re-allocated in loop() */
3
4  void handler(int sig)
5  {
6      /* printf is not async-signal-safe: locks may be held by
7       the interrupted thread => deadlock or torn state.
8       What must a handler use instead? */
9      printf("signal_%d, flushing_%s\n", sig, g_buf);
10 }
11
12 void loop(void)
13 {
14     for (;;) {
15         free(g_buf);
16         g_buf = malloc(0x100); /* handler cannot tell g_buf state */
17     }
18 }
```

RefCount Borrowing Bugs

1. The reference counting section covers over/under-decrement. A distinct family: borrowed references — pointers returned or stored without taking a refcount. The holder assumes the borrow is scoped to a lock, a call, or an ownership relationship that some later refactor silently broke.
2. Audit pattern: for every structure that stores another object's pointer without a get/put pair, construct the counterexample — what sequence of events frees the pointee while the borrower still holds it? Missing this pattern is how UAFs survive audits that checked the refcounts.

Listing 22: Borrowed Reference

```
1  /* verified: compiles cleanly at -O0; pattern intended */
2  struct Obj {
3      char name[0x20];
4      struct Obj *peer;    /* borrowed reference, no refcount taken */
5      int refcnt;
6  };
7
8  void link(struct Obj *a, struct Obj *b)
9  {
10     a->peer = b;    /* borrow without incrementing b->refcnt */
11 }
12
13 void drop(struct Obj *o)
14 {
15     if (--o->refcnt == 0) {
16         /* what do we owe anyone who borrowed o->peer? */
17         free(o);
18     }
19 }
```

Per-Pattern Summary

1. The same patterns, reduced to the tell an auditor greps for and the requirement an exploit has:

Pattern	Detection tell	Exploitation requirement
Stack overflow	fixed-size local buffer, length checked after or not at all	canary/ASLR bypass, a control-flow target (return address, saved pointer)
Heap overflow	allocation size and copy length from different sources	adjacency to allocator metadata or an interesting object; a groom
Format string	non-constant format argument to a <code>printf</code> family call	<code>%n</code> enabled, argument control, an info leak first
OOB read/write	index/offset not bounded by the true size	signed offset for relative-before; absolute pointer for arbitrary
Arbitrary free	attacker-influenced pointer reaching <code>free</code>	a live object at that address (a UAF)
Double free	same pointer freed twice, list entry not cleared	allocator's duplicate check bypassed, type confusion
UAF	free followed by a use of the stale pointer	same-size re-allocation with attacker data, then the use
Integer overflow	size arithmetic in a narrow type	allocation smaller than the subsequent copy
Signedness	negative value accepted by an unsigned check	bounds check defeated by the sign, then OOB
Type confusion	union/cast on an untrusted tag	layout mismatch at the use site (size, vtable, pointer type)
Info leak	output length exceeds initialized data	pointer or ASLR secret in the output
Command injection	unsanitized input reaching <code>system/popen</code>	shell metacharacter surviving the filter
Race (TOC-TOU)	check and use separated by a shared object	win the window, substitute the object
Logic error	misplaced <code>goto</code> , merge or copy-paste artifact	reachability of the branch that skips the check
RefCount	get/put imbalance, refcount wrap	premature free (UAF) or never-freed leak
Uninitialized	partially filled struct handed out	attacker-sprayed stale stack/heap content
Integer promotion	narrow operands promoted before compare	comparison semantics differ from the source
Truncation	wide value narrowed between API layers	check applied to the wide value, use of the narrow one
Deserialization	type tag and length validated separately	consumer trusts the tag-implied layout
Path/FD race	<code>lstat/open</code> split, fd reused later	substitute object between check and use
Signal safety	non-async-signal-safe call in a handler	deadlock or torn allocator state
RefCount borrow	pointer stored without a matching <code>get</code>	pointee freed while the borrower still lives

Beyond C: Recognizing Surfaces in Other Runtimes

Much of the industry's attack surface is not C. Recognition-level literacy in these environments prevents embarrassing mis-pricing ("that's just a webapp") and identifies when the same audit disciplines transfer.

Server-side web applications

1. The memory-safety story is replaced by a trust-boundary story. The recurring classes, each with its audit question:
 - (a) Injection family (SQL/OS/command/LDAP): user data concatenated into

a structured string an interpreter re-parses. Audit question: every place user data meets a string that a second interpreter sees — is it parameterized/bound or concatenated? (**Command Injection** above is the C-side member of this family; the SQL one is identical in shape with a different interpreter).

- (b) Path/IDOR (authorization by omission): object access checked by reachability, not ownership. Audit method: enumerate every object-ID in a request and, for each, ask “can user B read user A’s?” — privilege-model review, not parser review.
 - (c) SSRF: a server-side request on an attacker-influenced URL — audit every fetch/unserialize-target (the bug-bounty staple of the era).
 - (d) Session/auth: token predictability, “remember me” implementations, logout that leaves server-side state valid.
 - (e) Template/deserialization: template injection (SSTI) and naive object deserialization are the web’s type-confusion analogues — interpreter state shaped by the attacker.
2. Tooling notes: the source-tree habits transfer (grep for sink APIs — `query(...)`, `popen(...)`, `requests.get(...)`, template render calls); the map from input vector (HTTP verbs, headers, cookies) to handler code is the same attack-surface analysis, with routes as the fops table.
 3. When it merges back into this course: web servers on embedded devices (the AppWeb lab) expose both surfaces at once — HTTP parsing bugs in C and injection bugs in the modules they host.

Managed runtimes (Java/C#/Go) and scripting engines (JS/Python)

1. Memory corruption does not disappear; it migrates to the runtime’s own trusted computing base: interpreter bugs, JIT miscompilations (the fuzzing lecture’s Fuzzilli path), GC lifetime bugs (the refcount-borrow pattern in garbage-collected form), foreign-function-interface marshalling bugs (JNI, cgo — the C layer beneath the managed one).
2. *Type confusion* reappears as the engine’s own speculative-type assumptions (JS engines’ inline caches and JIT systems keyed on value shape) — the bug class behind a decade of browser CVEs.
3. Audit adaptation: the trusted layer (runtime, VM, standard library) is an attack surface of its own; the defenses (bounds checks) are inherited but the invariants frequently live in native extensions, JNI bridges, or allocator-connected libraries — audit the bridge, not the managed code.
4. Sandbox escape as a goal: when the runtime is itself the sandbox (browser renderer/render-process isolation; JVM plugin sandboxes), the audit target hierarchy is bridge bugs first (native code reachable from managed code), then runtime logic bugs (verifier/type-system flaws), then the substrate.

Exploitation Mitigations: a Per-Bug-Class Map

Understanding which mitigations oppose each bug class is what separates “I found a crash” from “I found an exploitable bug.” A bug’s exploitation cost is not a property of the bug alone but of the bug $\times$ the deployed mitigations.

Mitigation	Defeats / cripples	Bypassed by
NX / DEP	classic ret2-stack shellcode	ROP / JOP / ret2libc / ret2*
Stack canaries	indiscriminate return-address overwrites	infoleak of canary, OOB read before overwrite, non-cookie-guarded locals
ASLR	hardcoded addresses, non-relocatable payload links	infoleak, partial overwrites, brute force on 32-bit, side channels
RELRO / GOT hardening	GOT overwrite as final step	hooking .rodata function tables, overwriting libc structures
FORTIFY_SOURCE	classical memcpy/strcpy bounds misses	truncation arithmetic, off-by-one into adjacent fields
CFI / CFG	arbitrary function-pointer hijack	gadgets within valid dispatch sets, data-only attacks
MTE / ASan-class checks	near-miss OOB, UAF after quarantine	tag-stable re-allocations, within-region confusion

1. Reading the map: mitigations are cheap, per-bug-class shifts, not absolutes. Each “defeats/cripples” column entry is a lowering of bug reward, and the bypass column is the auditor’s mental checklist: every finding should be explicitly priced against each active mitigation on the target.
 - (a) CFI deployed: prioritize data-only corruption bugs (buffer content trusted as metadata, integer-length confusion). Function-pointer hijacks are worthless.
 - (b) RELRO incompletely deployed (partial): GOT overwrite is a first-class target; audit all call-out placements via got writes whenever a write primitive reaches the GOT.
 - (c) ASan-tagged target (test/CI builds): classic OOB gets caught immediately; fuzzing is weaker against distance-based logic bugs (races, state confusion). Adjust.
 - (d) Old target with no mitigations: classic ret2stack is back on the table; prioritize the stack-overflow pattern sections again.
2. How mitigations are wired in the compiler/OS is teachable but out of scope for this lecture; the fuzzing lecture covers sanitizer internals (ASan redzones/quarantine) which are the same map’s instrumentation arm.

From Bug to Report

1. When a candidate bug is found, the residual engineering follows a repeatable pipeline:
 - (a) **Minimize:** reduce the trigger to the smallest input/sequence demonstrating the fault (dd bisect, delta, or a purpose-built reproducer main()).

- (b) **Stabilize:** make it deterministic. Disable ASLR if needed, pin ordering, reproduce under both sanitizer and plain builds. A report that only reproduces under debug builds will be contested.
 - (c) **Root cause:** map the minimized trigger to a source-level explanation (which check is missing/wrong). If the explanation is not expressible in one or two sentences, the minimization is not done.
 - (d) **Classify:** which pattern class is it (the sections above)? What primitives do you actually get (arbitrary write? controlled length? partial control?).
 - (e) **Price the exploitability:** enumerate the active mitigations (map above), the attacker preconditions (local? network? authenticated?), and consequences (execution? data leak? DoS?). “Needs another bug to be exploitable” must be stated, not hidden.
 - (f) **Write the report:** one-paragraph description; minimized reproducer; exploitability statement; affected versions/builds; suggested fix. For a course deliverable, the reproducer must run unmodified on the unmodified vendor target.
2. The report is the profession’s unit of work, so the format is fixed. Every module’s lab ends in exactly this document:

Listing 23: From Bug to Report: report skeleton

```

1 Title:      <component> <bug class> <impact in one line>
2 Affected:   <version/build range, configuration options>
3 Summary:    one paragraph: what the code does, where it goes wrong,
4             what an attacker gets.
5 Reproducer: minimized input/sequence; exact build flags and
6             environment; observed result (sanitizer output or crash).
7 Root cause: one or two sentences naming the missing/wrong check,
8             with the source location.
9 Exploitability: the primitive (from the vocabulary), the active
10             mitigations and their bypass cost, the attacker
11             preconditions, and what else is required.
12 Fix:       suggested change, plus the test that would have caught it.
```

1. **Common report defects** (the reasons a finding is rejected, and therefore the things to get right the first time):
- (a) Non-reproducible: no exact build, no flags, no environment. Attach all three.
 - (b) Unexplained root cause: the report describes the crash, not the bug. If the missing check cannot be named, minimization is not finished.
 - (c) Overpriced exploitability: the report claims arbitrary write where only a controlled-length write exists. Under-claiming is safer than over-claiming; the map above prices it.
 - (d) Out-of-scope target: check the vendor’s or platform’s scope before investing the pipeline.

2. Disclosure considerations (orientation only, not legal advice):

- (a) Coordinated vs. full disclosure: coordinated (report privately, publish on fix or after a deadline) is the professional default. 90-day is the widely referenced policy; vendors' own SSRP/CSRP/similar policies have their own clocks and scope rules.
 - (b) Bug bounty platforms carry their own scope: check what's in and out of scope before spending effort — reject findings that will be closed as “out-of-scope” regardless of quality.
 - (c) Keep reproducer material inside the reporting chain. Publishing working exploits without coordination both harms users and removes your own leverage.
3. Exercise: for any two examples in this lecture, run the full pipeline on paper: write the one-paragraph report as if the reproducer ran on an old AppWeb version, complete with exploitability pricing.

Source Auditing Tools

1. Source auditing tools can improve productivity considerably by reducing time to perform attack surface analysis, find definitions, locate calling functions, and understanding of logical flow.
2. The productivity argument is the same one the reversing module makes: an auditor's time is the scarce resource, so the tool's job is to rank what to read. Any tool that does not shorten the distance from an input vector to a candidate sink is not worth the setup.
3. Critical features of any source auditing tool are as follows.
 - (a) Navigating cross references (ctags, cscope)
 - i. Searching for definitions of functions, classes, structs, types, etc
 - ii. Searching for places a function is called or referenced
 - iii. The reverse dependency (who calls this) is what makes a sink reachable from an entry point; the forward dependency (what does this call) is what turns a candidate into a primitive.
 - (b) Understanding preprocessor directives and expand complicated macros: a large part of the real program exists only after `#ifdef` processing, so audit the configuration the target actually ships with.
 - (c) Search by file name (sublime text)
 - (d) Search by struct or object type (eclipse)
 - (e) Meld is a handy tool to diff source files, which is also how a vendor patch is read backwards into a bug.
 - (f) Joern is a newer tool that allows for writing complicated queries to find vulnerable patterns in abstract syntax trees: the grep-based habits above expressed as queries over a code property graph, so that a data-flow question (does input reach this sink?) is answered mechanically.

- (g) Visual Studio, Eclipse and Understand are all excellent source tools. Only Eclipse is free; the others are paid.
- (h) Whatever the tool, the output is a set of candidates, never a set of findings. The three questions from the audit method above are still the auditor's, and the false-positive rate of every automated tool is the reason.

Conclusion

Understanding code well enough to find bugs is a difficult problem, which is why code reviews are still the most effective way for developers to avoid releasing buggy code. The most advanced automated source analysis tools, such as those from Grammat-ech, employ constraint solving and code simulation environments. However, all such tools suffer from high false-positive rates: the tool can show that a path exists, but not that an attacker controls the data on it, and not what primitive the corruption grants. Understanding which code is relevant in an attack scenario and what constitutes an exploitable bug is difficult to do in an automated fashion.

The conclusion for the auditor is a division of labor. Automation enumerates the surface (which entry points exist, which functions are reachable, which sinks are near attacker data); the human answers the three questions above and prices the result against the mitigation map. The rest of this course is that division of labor applied to the other families: fuzzing replaces the reachability half with execution, reversing replaces the source half when there is no source, debugging converts every hypothesis into an observation, and the exploitation module turns the primitives named above into working chains.

References

1. blog.nsogroup.com, A tale of two mallocs
2. cscope.sourceforge.net
3. ctags.sourceforge.net
4. exploit-exercises.lains.space
5. joern.io (code property graph queries)
6. meldmerge.org
7. pleasestopnamingvulnerabilities.com/integers.html
8. scitools.com (understand tool)
9. visualstudio.microsoft.com
10. www.eclipse.org
11. www.fuzzingbook.com
12. www.grammatech.com (codesonar tool)

13. clang static analyzer (clang-analyzer.llvm.org), Coverity, CodeQL and Semgrep (semgrep.dev) as the automated-scanning complement to the manual audit
14. A Bug Hunter's Diary, Book by Tobias Klein
15. The Art of Software Security Assessment, Book by John McDonald, Justin Schuh, and Mark Dowd